

Modelos Generativos Profundos

Clase 11: Redes generativas adversarias (parte II)

Fernando Fêtis Riquelme

Primavera, 2025

Facultad de Ciencias Físicas y Matemáticas

Universidad de Chile

Redes convolucionales y convolución transpuesta

DCGAN y U-Net para segmentación

Redes convolucionales y convolución transpuesta

Redes neuronales convolucionales

Si se utilizan redes MLP para procesar imágenes, entonces cada pixel de la imagen se relaciona del mismo modo con todos los otros pixeles, por lo que no se aprovecha la estructura espacial 2D de las imágenes. Las **redes neuronales convolucionales (CNNs)** utilizan **filtros convolucionales**, lo que induce:

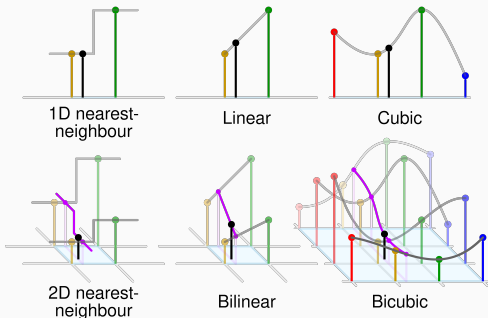
- **Sesgo de localidad:** cada neurona solo ve una vecindad local de la entrada (campo receptivo).
- **Equivarianza traslacional:** un filtro convolucional detecta el mismo patrón sin importar dónde se encuentre en la imagen.

Recordar que, usualmente, las CNNs van reduciendo la resolución de los mapas de características al mismo tiempo que aumentan la cantidad de canales.

Redes neuronales convolucionales

En IA generativa, es usual generar una imagen aumentando progresivamente la resolución de una representación latente inicial. Para esto, se pueden usar dos enfoques:

- Upsampling por interpolación.
- Convolución transpuesta.



Operación convolución y convolución transpuesta

Para entender mejor el operador convolución (más correctamente, cross-correlation), se tiene el siguiente ejemplo de una convolución 1D para un filtro $K = [5, 10, 15, 20]$:

Dada una entrada $x = (1, 2, 3, 4, 5, 6, 7, 8, 9, 10)^T \in \mathbb{R}^{10}$, la salida de la convolución (asumiendo $\text{stride} = 2$ y $\text{padding} = 0$) es el vector $y \in \mathbb{R}^4$ cuyas coordenadas son

$$y_1 = 5 \cdot 1 + 10 \cdot 2 + 15 \cdot 3 + 20 \cdot 4 = 150$$

$$y_2 = 5 \cdot 3 + 10 \cdot 4 + 15 \cdot 5 + 20 \cdot 6 = 250$$

$$y_3 = 5 \cdot 5 + 10 \cdot 6 + 15 \cdot 7 + 20 \cdot 8 = 350$$

$$y_4 = 5 \cdot 7 + 10 \cdot 8 + 15 \cdot 9 + 20 \cdot 10 = 450$$

Operación convolución y convolución transpuesta

La convolución es una **transformación lineal**, por lo que puede representarse como una multiplicación matricial de la forma $x \mapsto Wx$, donde W es una matriz que depende del filtro convolucional (y de otros parámetros como el stride y padding).

En particular, para el ejemplo anterior, se tiene que $y = Wx$, donde $W \in \mathcal{M}_{4,10}(\mathbb{R})$ es la **matriz de Toeplitz** definida como

$$W = \begin{pmatrix} 5 & 10 & 15 & 20 & 0 & 0 & 0 & 0 & 0 & 0 \\ 0 & 0 & 5 & 10 & 15 & 20 & 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 & 5 & 10 & 15 & 20 & 0 & 0 \\ 0 & 0 & 0 & 0 & 0 & 0 & 5 & 10 & 15 & 20 \end{pmatrix}$$

En convoluciones 2D también es posible hacer esto. En estos casos, la matriz W tiene una estructura más compleja, pero sigue siendo posible definirla.

Operación convolución y convolución transpuesta

Dada una convolución representada por $x \in \mathbb{R}^n \mapsto Wx \in \mathbb{R}^m$ (con $W \in \mathcal{M}_{m,n}(\mathbb{R})$), la **convolución transpuesta** es el operador asociado a la transformación lineal $x \in \mathbb{R}^m \mapsto W^T x \in \mathbb{R}^n$.

Si la convolución reduce la dimensión de la entrada, la convolución transpuesta la aumenta, por lo que esta operación puede interpretarse como una especie de *convolución inversa*.

En el ejemplo anterior, la convolución mapeaba vectores de $\mathbb{R}^{10}$ a vectores de $\mathbb{R}^4$, mientras que la convolución transpuesta mapea vectores $x \in \mathbb{R}^4$ a vectores de $y \in \mathbb{R}^{10}$.

Esto se extiende naturalmente a convoluciones 2D. En consecuencia, la convolución transpuesta permite aumentar la resolución espacial de los mapas de características en una CNN.

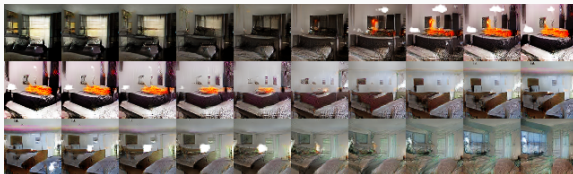
DCGAN y U-Net para segmentación

La naturaleza adversativa de una GAN provoca que su entrenamiento sea muy inestable. El trabajo de **deep convolutional GAN (DCGAN)** entrega algunas heurísticas para usar CNNs en el generador y en el discriminador de una GAN:

- El discriminador es una CNN clásica (sin convoluciones transpuestas) y el generador es una CNN transpuesta.
- Usar BatchNorm en todas las capas del generador y discriminador, excepto en la capa de entrada del discriminador y la capa de salida del generador.
- Usar ReLU en todas las capas del generador, excepto en la salida donde se recomienda utilizar `tanh`. Pero usar LeakyReLU en todas las capas del discriminador.
- etc.

DCGAN también estudia la estructura algebraica que emerge en el espacio latente de la GAN: ciertas operaciones vectoriales sobre representaciones latentes se traducen en transformaciones coherentes en el espacio de las imágenes generadas.

Un ejemplo es la **interpolación semántica**. Si $z_0, z_1 \in \mathbb{R}^L$ son dos muestras de la variable latente $p(z) \sim \mathcal{N}(0, I_L)$, entonces es posible construir una interpolación latente entre las muestras $G_\theta(z_0), G_\theta(z_1) \in \mathbb{R}^D$ considerando $x_t = G_\theta((1 - t)z_0 + tz_1) \in \mathbb{R}^D$, donde $t \in [0, 1]$ es un parámetro de interpolación.



La estructura vectorial del espacio latente $\mathbb{R}^L$ también permite hacer otro tipo de operaciones. Para esto, considerar:

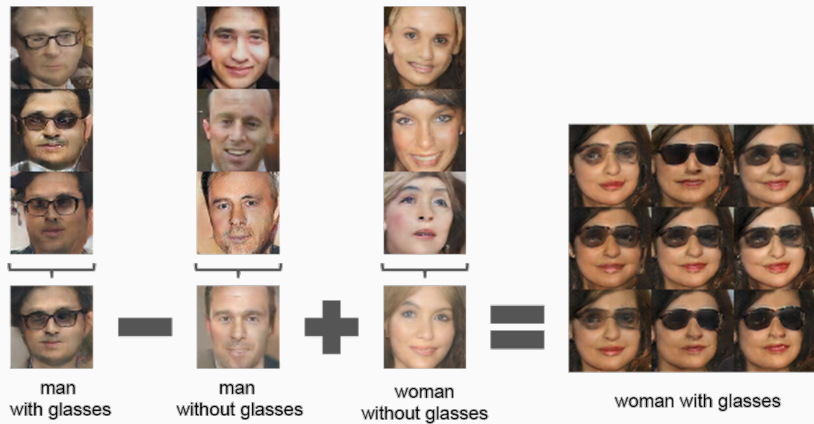
- $z_{\text{man, glasses}} \in \mathbb{R}^L$ es un promedio de vectores latentes que generan hombres con lentes.
- $z_{\text{man, no glasses}} \in \mathbb{R}^L$ es lo mismo pero genera hombres sin lentes.
- $z_{\text{woman, no glasses}} \in \mathbb{R}^L$ es lo mismo pero genera mujeres sin lentes.

entonces el vector latente (e incluso una vecindad de él)

$$z_{\text{woman, glasses}} = z_{\text{man, glasses}} - z_{\text{man, no glasses}} + z_{\text{woman, no glasses}}$$

genera una imagen de una mujer con lentes.

DCGAN



Arquitectura U-Net

Las CNNs y las CNN transpuestas pueden combinarse en una misma arquitectura para así obtener una arquitectura que reciba una imagen y entregue otra imagen como salida.

Una forma natural es usar un **autoencoder convolucional**:

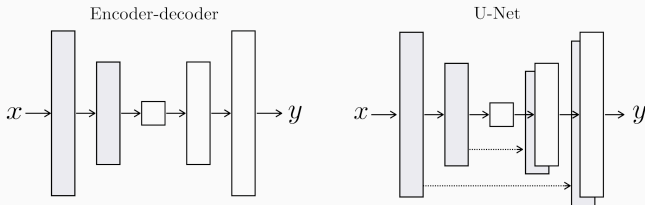
- Una CNN reduce progresivamente la resolución de la imagen de entrada hasta obtener una representación latente.
- Una CNN transpuesta aumenta progresivamente la resolución de la representación latente hasta obtener la imagen de salida.

Sin embargo, esta arquitectura tiene problemas para preservar detalles finos de la imagen de entrada, lo cual es relevante en tareas como la **segmentación de imágenes** o **image-translation**.

La **arquitectura U-Net** mejora esto agregando **skip-connections** entre el encoder y el decoder.

Arquitectura U-Net

Si bien esta arquitectura fue propuesta originalmente para segmentación de imágenes biomédicas, es muy usada en IA generativa, tanto para GANs como para modelos de difusión.



- En la práctica, se suele utilizar una versión con bloques ResNet y mecanismos de atención.
- En modelos de difusión, esta arquitectura ha sido desplazada por arquitecturas tipo DiT (redes tipo ViT para difusión).

En la próxima clase.

- Otras arquitecturas neuronales para GANs.
- FID.
- Transferencia de estilo.

Modelos Generativos Profundos

Clase 11: Redes generativas adversarias (parte II)